

Computer Organization-1

Study Guide

Prof. Saurav Bharadwaj
Assistant Professor
Dept of ECE, PIET
Parul University

1. Register Transfer language
2. Bus design using multiplexer and Tri-state buffer
3. Memory Transfers
4. Arithmetic Micro- Operations
5. Logic Micro-Operations
6. Shift Micro-Operations
7. Arithmetic logical shift unit.
8. Instruction codes
9. Computer registers
10. Computer instructions
11. Timing and Control
12. Instruction cycle
13. Memory- Reference Instructions
14. Register Reference Instructions
15. IO Reference Instructions
16. Interrupt
17. Design of Accumulator Unit

5.1 Register Transfer language

Register Transfer Language (RTL) in the context of the 8085 microprocessor is a symbolic notation used to describe how data moves between registers, memory, and the ALU (Arithmetic and Logic Unit) during the execution of instructions. It is a low-level description of operations in terms of micro-operations (basic data movements or processing steps). RTL is a symbolic notation used to describe the operations, data transfers, and manipulations that happen within the processor at the register level.

- RTL expresses the internal operations of the CPU.
- It shows how data is transferred between various components like registers (e.g., A, B, C, D, E, H, L), memory, and internal buses.
- It also includes control signals that trigger these data movements.

Data Transfer Operations: These involve the movement of data from one location (register/memory) to another without any change to the data.

Examples in RTL:

- $A \leftarrow B$: Transfer the contents of register B into register A.
- $M \leftarrow A$: Transfer the contents of the accumulator to memory (M).
- $A \leftarrow M$: Transfer memory content to accumulator.

Examples in 8085 Instructions:

- MOV A, B
- MOV M, A
- MOV A, M

Arithmetic Operations: These involve arithmetic computations such as addition or subtraction.

Examples in RTL:

- $A \leftarrow A + B$: Add contents of B to A.
- $A \leftarrow A - C$: Subtract contents of C from A.

Examples in 8085 Instructions:

- ADD B
- SUB C

Logical Operations: These include logical operations like AND, OR, XOR, NOT, etc.

Examples in RTL:

- $A \leftarrow A \wedge B$: Logical AND of A and B.
- $A \leftarrow A \vee C$: Logical OR of A and C.

- $A \leftarrow A \oplus D$: Logical XOR of A and D.
- $A \leftarrow \neg A$: Complement the contents of A.

Examples in 8085 Instructions:

- ANA B (AND A with B)
- ORA C (OR A with C)
- XRA D (XOR A with D)
- CMA (Complement Accumulator)

Shift and Rotate Operations: These shift or rotate bits in a register, typically the accumulator.

Examples in RTL:

- $A \leftarrow A \ll 1$: Shift accumulator left.
- $A \leftarrow A \gg 1$: Shift accumulator right.
- $A \leftarrow \text{RLC}(A)$: Rotate left.
- $A \leftarrow \text{RRC}(A)$: Rotate right.

Examples in 8085 Instructions:

- RLC (Rotate left)
- RRC (Rotate right)
- RAL, RAR (Rotate through carry)

Branching/Control Transfer Operations: These control the flow of execution based on certain conditions.

Examples in RTL:

- $\text{PC} \leftarrow \text{address}$: Jump to the specified address.
- If $(Z = 1)$ then $\text{PC} \leftarrow \text{address}$: Conditional jump if zero flag is set.

Examples in 8085 Instructions:

- JMP 2000H
- JZ 3000H (Jump if zero flag is set)
- CALL 2050H, RET

Flag Manipulation: Operations that directly affect flag registers.

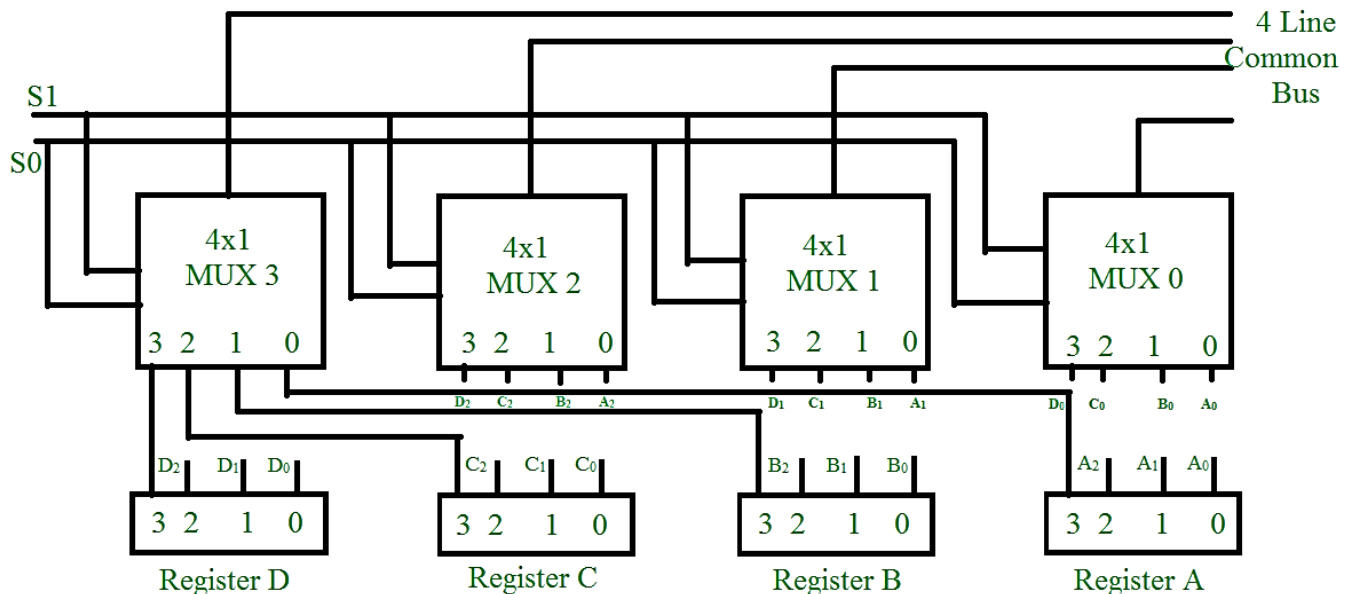
Examples in RTL:

- $Z \leftarrow 1$: Set Zero flag.
- $CY \leftarrow 0$: Clear Carry flag.

Examples in 8085 Instructions:

- STC (Set Carry)
- CMC (Complement Carry)
- DAA (Decimal Adjust Accumulator)

5.2 Bus design using multiplexer and Tristate buffer



- Bus design using multiplexers (MUX) and tristate buffers is a crucial aspect of digital circuit design, particularly when it comes to efficiently managing data flow in a shared bus system. A bus is essentially a set of physical connections used for transmitting data between different components of a system, such as between a processor and memory or between multiple peripherals.
- A multiplexer is a combinational logic circuit that selects one of many input signals and forwards the selected input to a single output line. A multiplexer has multiple data input lines, a single output line, and control lines (or selection lines) that determine which input is connected to the output.
- **Characteristics of Multiplexers:**
 - N-input MUX: A multiplexer with n data input lines will require $\log_2(n)$ control lines to select one of the inputs.
 - A 2-to-1 MUX has two input lines, one output line, and one selection line. The output is connected to one of the input lines based on the selection signal.
 - A 4-to-1 MUX has four input lines, one output line, and two selection lines, etc.
- In bus systems, multiplexers are used to manage data from different sources onto a single bus. This ensures that only one component can drive the bus at a time, avoiding conflicts. The multiplexer allows different devices (e.g., CPU, memory, peripherals) to send data over the same bus by selecting which source is connected to the bus at any given time.
- The tristate buffer is critical in bus systems because multiple components can share a bus, and only one component should be allowed to drive the bus at any given time. When a tristate buffer is in the high-impedance state (Z), it effectively "disconnects" itself from the bus, allowing other components to use the bus without interference.

- **Role in Bus Design:**

- Controlled Access: Tristate buffers ensure that only one device can drive the bus at any given time. Devices not actively driving the bus are placed in the high-impedance state to avoid contention.
 - Simplifies Design: Tristate buffers simplify the design by eliminating the need for complex switching mechanisms to disconnect or connect devices to the bus.
- The construction of this bus system for 4 registers is shown above. The bus consists of 4×1 multiplexers with 4 inputs and 1 output and 4 registers with bits numbered 0 to 3. There are 2 select inputs S0 and S1 which are connected to the select inputs of the multiplexers.
- The output 1 of register A is connected to input 0 of MUX 1 and similarly other connections are made as shown in the diagram. The data transferred to the bus depends upon the select lines.

Feature	Multiplexer (MUX)	Tri-state Buffer
Main Function	Selects one input from many and forwards it to output	Connects a device to a shared bus or disconnects it
Control Signal	Uses select lines to pick the input	Uses an enable signal to connect/disconnect output
Output Behavior	Always outputs one selected input	Output can be 0, 1, or High-Impedance (Z)
Connection Style	One device is connected via MUX logic	All devices are physically connected, but only one drives the bus
Use Case	Useful when data selection is needed (e.g., choosing data source)	Useful for bus systems with many drivers (e.g., memory, registers)
Scalability	Harder to manage with many inputs	Easier to add more devices
Example	4-to-1 MUX selects 1 input out of 4	A register drives a bus only when its enable is active

- **Multiplexers in Bus Design:** Multiplexers are used to select which source's data gets placed on the bus. A multiplexer is typically controlled by control signals that determine which input (representing different data sources) should be sent to the bus. For example, in a system with multiple peripherals and a processor, the processor, each peripheral, and the memory can be connected to a bus through multiplexers. The processor can use control lines to select whether the data comes from the memory or the peripherals, and that data gets routed onto the bus.
- **Tristate Buffers in Bus Design:** Each device connected to the bus will have its output connected to a tristate buffer. The control logic ensures that only one device can drive the bus at any time. When a device is not selected to place data on the bus, its tristate buffer is placed in the high-impedance state, effectively disconnecting it from the bus. The address lines, data lines, and control lines of devices can be linked to the bus through tristate buffers, with the control lines determining when each device is allowed to place data on the bus.

- **Combined Operation:**

- **Selection of Data Source:** The multiplexer selects which data source (memory, I/O, processor, etc.) will be connected to the bus at any time.
- **Enabling the Data Source:** The tristate buffer associated with the selected device is enabled, allowing it to drive the bus.
- **Disabling Other Devices:** The tristate buffers for the non-selected devices are placed in the high-impedance state (Z), preventing any data conflict on the bus.

5.4 Arithmetic Micro-Operations

Arithmetic micro-operations are fundamental operations performed on binary numbers stored in the registers of a digital computer. These operations include addition, subtraction, increment, decrement, and various arithmetic manipulations like two's complement. The primary goal of arithmetic micro-operations is to perform basic mathematical functions at the micro-level of a processor's control unit.

The most common arithmetic micro-operation is binary addition, which adds two binary numbers and may generate a carry. Similarly, subtraction is often implemented by adding the two's complement of the subtrahend to the minuend. For example, to subtract B from A ($A - B$), the system takes the two's complement of B and adds it to A. Other operations include incrementing (adding 1) or decrementing (subtracting 1) a register value, which are useful in counting or looping processes. These operations are crucial for executing high-level instructions like arithmetic calculations in programs.

Operation	RTL Notation	Meaning
Transfer	$R1 \leftarrow R2$	Copy the contents of register R2 into R1.
Addition	$R1 \leftarrow R2 + R3$	Add contents of R2 and R3 and store the result in R1.
Subtraction	$R1 \leftarrow R2 - R3$	Subtract contents of R3 from R2 and store in R1.
Increment	$R1 \leftarrow R1 + 1$	Add 1 to the contents of R1.
Decrement	$R1 \leftarrow R1 - 1$	Subtract 1 from the contents of R1.
Add with Carry	$R1 \leftarrow R2 + R3 + C_{in}$	Add R2, R3, and carry-in bit (C_{in}).
Subtract with Borrow	$R1 \leftarrow R2 - (R3 + 1)$	Subtract R3 and borrow bit from R2.
Two's Complement	$R1 \leftarrow -R2$	Take two's complement of R2 and store in R1 (negate).
Clear	$R1 \leftarrow 0$	Set all bits of R1 to 0.

5.5 Logic Micro-Operations

Logic micro-operations deal with the bitwise manipulation of binary data using logical operations. These operations include AND, OR, XOR (exclusive OR), and NOT (complement). Unlike arithmetic operations

that deal with numerical calculations, logic operations focus on the manipulation of individual bits based on Boolean logic.

For example, the AND operation compares two bits and returns 1 only if both bits are 1. The OR operation returns 1 if either of the bits is 1, while the XOR returns 1 only if the bits differ. The NOT operation (also called complement) inverts the bits — changing 0 to 1 and vice versa. These operations are typically used in tasks like setting, clearing, toggling specific bits in a register, or for masking operations where specific bits are filtered out. Logic micro-operations form the foundation of decision-making processes and control flow within the processor.

Operation	Symbol	Meaning
AND	$\wedge$ or AND	Bitwise AND
OR	$\vee$ or OR	Bitwise OR
XOR	$\oplus$ or XOR	Bitwise Exclusive-OR
NOT	$\neg$ or NOT	Bitwise Complement

RTL Statement	Meaning
$R2 \leftarrow R1 \text{ AND } R3$	Perform bitwise AND between R1 and R3, store result in R2
$R2 \leftarrow R1 \text{ OR } R3$	Perform bitwise OR between R1 and R3, store result in R2
$R2 \leftarrow R1 \text{ XOR } R3$	Perform bitwise XOR between R1 and R3, store result in R2
$R2 \leftarrow \text{NOT } R1$	Perform bitwise NOT (complement) of R1, store in R2

5.6 Shift Micro-Operations

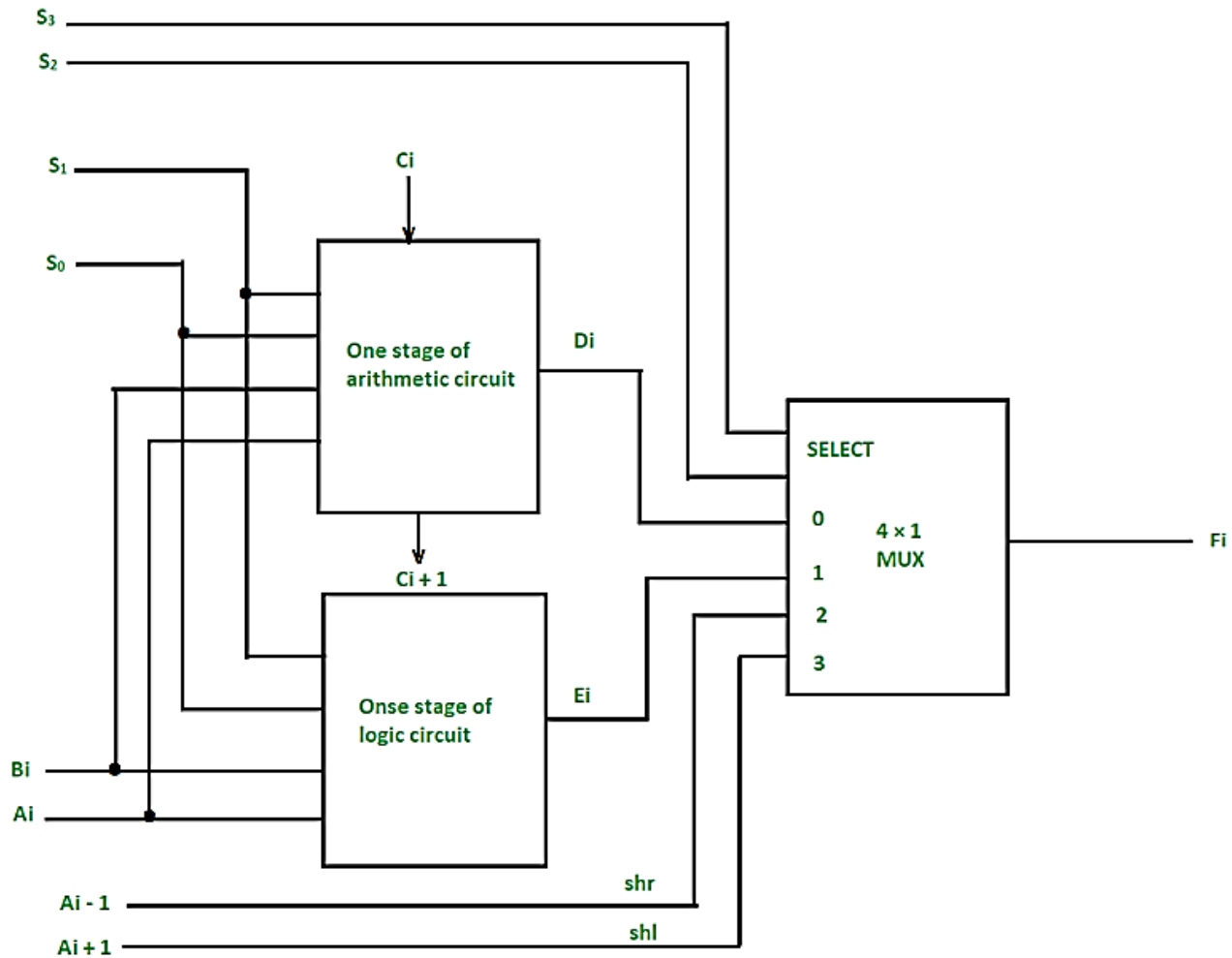
Shift micro-operations involve moving the bits in a register to the left or right. These operations are essential for tasks like multiplication or division by powers of two, or for data alignment. The three main types of shift micro-operations are logical shift, arithmetic shift, and circular (or rotate) shift.

A logical shift moves the bits left or right, inserting zeros into the vacated bit positions. A logical left shift moves all bits one position to the left and places a 0 in the least significant bit (LSB), effectively multiplying the binary number by 2. Conversely, a logical right shift moves bits to the right and inserts a 0 in the most significant bit (MSB), effectively dividing the number by 2. An arithmetic shift, on the other hand, preserves the sign of signed binary numbers. An arithmetic right shift replicates the sign bit (MSB) instead of inserting a zero, thus maintaining the number's sign during division. Finally, a circular shift (rotate shift) wraps the bits around so that no information is lost — for example, in a left rotate, the MSB becomes the LSB. These operations are vital in cryptography, signal processing, and low-level data manipulation.

Operation	RTL Representation	Description
Logical shift left	$R \leftarrow R \ll 1$	Shift all bits left; insert 0 into LSB.
Logical shift right	$R \leftarrow R \gg 1$	Shift all bits right; insert 0 into MSB.
Arithmetic shift right	$R \leftarrow \text{ashR}(R)$	Shift right; copy MSB into new MSB.
Rotate left	$R \leftarrow \text{ROL } R$	Rotate bits left; MSB to LSB.
Rotate right	$R \leftarrow \text{ROR } R$	Rotate bits right; LSB to MSB.

5.7 Arithmetic Logic Shift Unit (ALSU)

- Arithmetic Logic Shift Unit (ALSU) is a member of the Arithmetic Logic Unit (ALU) in a computer system. It is a digital circuit that performs logical, arithmetic, and shift operations.
- Rather than having individual registers calculating the micro operations directly, the computer deploys a number of storage registers which is connected to a common operational unit known as an arithmetic logic unit or ALU.
- The Arithmetic Logic Unit performs an operation that leads as a result and gets transferred to a destination register.
- Arithmetic Logic Unit may be a combinatory circuit in order that the complete register transfer operation from the supply registers through the ALU and into the destination register is performed throughout one clock pulse amount.
- Sometimes, the shift micro operations are performed in a separate unit, but sometimes it is made as a part of full ALU.

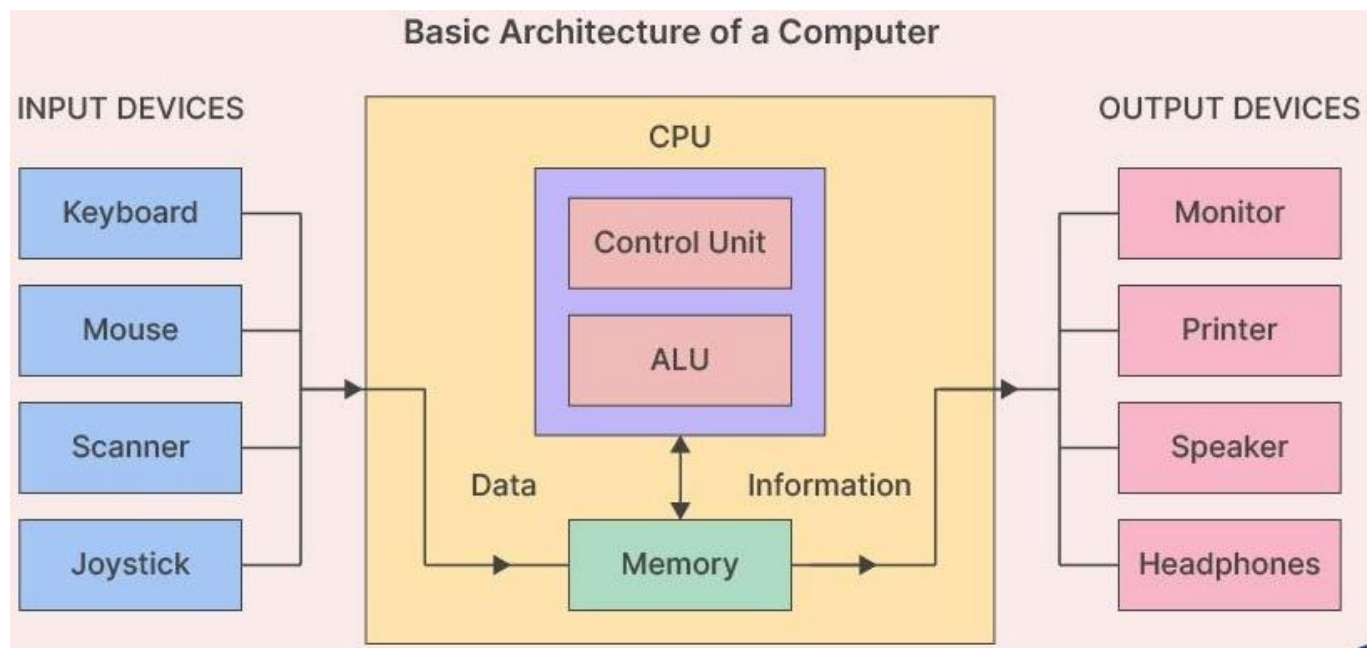


- 4 x 1 multiplexer at the output chooses between associate arithmetic output between Ei and a logic output in Hi. The data in the multiplexer are selected through inputs S3 and S2 and the other two data inputs to the multiplexer obtain the inputs Ai – 1 for the shr operation and Ai + 1 for the shl operation.

Operation Select					Operation	Function
S ₃	S ₂	S ₁	S ₀	C _{in}		
0	0	0	0	0	F = A	Transfer A
0	0	0	0	1	F = A + 1	Increment A
0	0	0	1	0	F = A + B	Addition
0	0	0	1	1	F = A + B + 1	Add with carry
0	0	1	0	0	F = A + B'	Subtract with borrow
0	0	1	0	1	F = A + B' + 1	Subtraction
0	0	1	1	0	F = A - 1	Decrement A
0	0	1	1	1	F = A	Transfer A
0	1	0	0	x	F = A ^ B	AND
0	1	0	1	x	F = A v B	OR
0	1	1	0	x	F = A XOR B	XOR
0	1	1	1	x	F = A'	Complement A
1	0	x	x	x	F = shr A	Shift right A into F
1	1	x	x	x	F = shl A	Shift left A into F

The circuit whose one stage is given in the below diagram provides 8 arithmetic operations, 4 logic operations, and 2 shift operations, and Each operation is selected by the 5 variables S3, S2, S1, S0, and Cin.

5.8 Instruction codes



- **Input Unit:** Devices like a keyboard or mouse that feed data into the computer. The input unit comprises devices that are used to feed data and instructions into a computer system for processing. Common input devices include the keyboard, which allows users to type in textual data and commands, and the mouse, which lets users interact with graphical elements on the screen. Other input devices include scanners, microphones, webcams, and touchscreens. These devices convert human-readable data into a digital format that the computer can interpret and process. The input unit acts as the gateway for user interaction with the machine, ensuring that the data reaches the system's memory or processor accurately and efficiently.
- **Output Unit:** Devices like a monitor or printer that show results. The output unit is responsible for presenting the results of the computer's processes to the user. After the CPU completes its operations on the input data, the output unit conveys the results through various output devices. Monitors (or display screens) visually present information in the form of text, images, and videos, while printers produce hard copies of digital documents. Other output devices include speakers (which output sound) and projectors. The output unit effectively translates the computer's digital results into a form that is comprehensible and useful to users.
- **Memory Unit:** Stores data and instructions. The memory unit is a critical component of the computer that stores data, instructions, and intermediate results required for processing. It is broadly categorized into primary and secondary memory.
 - Primary Memory includes RAM (Random Access Memory) and ROM (Read-Only Memory).

RAM is volatile memory that temporarily holds data and instructions currently in use, enabling fast access by the CPU. Once the computer is turned off, the contents of RAM are lost. ROM, on the other hand, is non-volatile and contains permanent instructions needed to start the computer (such as firmware).

- Secondary Memory provides long-term storage and includes devices like hard disk drives (HDDs) and solid-state drives (SSDs). Unlike primary memory, secondary storage retains data even when the power is off. It is used to store the operating system, applications, and user data for future use.
- Central Processing Unit (CPU): The "brain" of the computer. Often referred to as the "brain" of the computer, the Central Processing Unit (CPU) is the core component responsible for executing instructions and managing tasks. It processes data retrieved from memory and controls the operation of all parts of the computer system. The CPU is divided into two main subcomponents:
 - The Control Unit (CU) manages and directs the operations of the processor. It interprets instructions from the program and signals other parts of the system—such as the memory and input/output devices—on how to respond.
 - The Arithmetic Logic Unit (ALU) is responsible for performing all arithmetic calculations (like addition and subtraction) and logical operations (such as comparisons). The CPU's performance and speed largely determine the overall efficiency of a computer system.
- Bus: The bus is a communication system within the computer that transfers data between the CPU, memory, and other hardware components. It acts like a highway for data, enabling different parts of the computer to communicate with one another. There are three primary types of buses:
 - The data bus carries the actual data being processed.
 - The address bus carries the memory addresses of where data is located or where it should be sent.
 - The control bus carries control signals that manage and coordinate the operations of the computer components.
 - Together, these buses ensure smooth and synchronized communication across the system, making it possible for the CPU to interact effectively with memory and input/output devices.

5. 9 Computer registers

- The 8085 has six general-purpose registers: B, C, D, E, H, and L. Each of these registers is 8 bits wide, but they can also be paired to perform 16-bit operations. The valid register pairs are BC, DE, and HL. These registers are primarily used to store data temporarily during program execution. For example, a programmer might use register B to store an operand and register C to store another, allowing a computation or data transfer operation. When used in pairs (e.g., HL), these registers can hold 16-bit memory addresses or 16-bit data values, which expands their utility in handling more complex tasks.
- The Accumulator (A) is a special-purpose register that plays a central role in the 8085's arithmetic and logical operations. Most of the 8085's instructions operate using the accumulator as one of the operands. It temporarily stores the results of operations like addition, subtraction, logical AND/OR, etc. Alongside the accumulator is the Flag Register, which reflects the outcomes of operations

performed by the ALU (Arithmetic Logic Unit). The flag register is an 8-bit register, but only five of its bits are used as status flags: Sign (S), Zero (Z), Auxiliary Carry (AC), Parity (P), and Carry (CY). These flags are automatically set or cleared depending on the result of an operation, and they are critical for decision-making in program execution, such as conditional branching.

- Two other important 16-bit registers in the 8085 are the Stack Pointer (SP) and the Program Counter (PC). The Stack Pointer is used to manage the stack—a special area in memory used for storing return addresses, local variables, and for saving the state of the processor during subroutine calls and interrupts. The Program Counter, on the other hand, keeps track of the memory address of the next instruction to be executed. After fetching an instruction, the PC automatically increments to point to the subsequent instruction. This register is crucial for ensuring the sequential execution of instructions in a program.
- Internally, the 8085 also uses a Temporary Register, which is not accessible to the programmer. It is used by the microprocessor during operations like arithmetic and logical processing to hold intermediate results. Similarly, the Instruction Register is used internally to hold the opcode of the current instruction being decoded and executed. These registers work behind the scenes to support the functioning of the CPU without direct programmer control.

5.10 Computer instructions

- **Memory-reference instructions** are the most common type of instructions in a computer's instruction set. These instructions typically involve accessing data stored in the system's memory. To perform operations like reading a value, modifying it, or writing it back, the instruction must specify a memory address where the data is located. Examples of memory-reference instructions include loading data from memory into a register, storing data from a register back to memory, and performing arithmetic operations directly involving memory locations. Because so much of a computer's work involves moving and manipulating data stored in memory, these instructions are fundamental to system operation.
- **Register-reference instructions** are designed to operate directly on the CPU's internal registers rather than external memory. Registers are small, very fast storage locations within the CPU used to hold temporary data and intermediate results. Instructions that work directly with registers are typically faster than memory-reference instructions because they avoid the time-consuming process of accessing external memory. Examples include operations like clearing a register, complementing its content, shifting its bits, or performing simple arithmetic operations between registers. These instructions are essential for speeding up computations and efficient CPU operations.
- **Input-output (I/O) instructions** handle the communication between the CPU and external devices such as keyboards, monitors, printers, and disk drives. Rather than operating on memory or internal registers, these instructions are designed to read data from an input device or send data to an output device. I/O instructions are crucial for making the computer interact with the outside world, allowing users to input data for processing and receive results after processing. Specialized I/O control instructions are used to manage the transfer of information, handle device status checking, and coordinate device communication without interfering with the main computational work.

5.11 Timing and Control

The 8085 microprocessor, developed by Intel, is an 8-bit microprocessor that operates based on clock-driven operations. One of the essential aspects of understanding the functioning of the 8085 microprocessor is the Timing and Control mechanism, which governs how instructions are fetched, decoded, and executed in coordination with memory and I/O devices. This mechanism ensures the synchronization of internal and external operations with the microprocessor's clock.

Timing in the 8085 refers to the coordination of various operations through clock pulses. The microprocessor uses a crystal oscillator to generate a system clock, which determines the speed at which the processor operates. Each operation in the 8085 is divided into machine cycles and T-states. A machine cycle is the time required to complete one basic operation such as opcode fetch, memory read, memory write, I/O read, or I/O write. Each machine cycle consists of one or more T-states (clock cycles). For example, an opcode fetch cycle typically requires 4 to 6 T-states, while a memory read or write cycle may take 3 T-states.

There are different types of machine cycles in 8085, such as:

- **Opcode Fetch Cycle (OF):** The processor fetches the instruction from memory.
- **Memory Read/Write Cycle (MR/MW):** The processor reads from or writes to a memory location.
- **I/O Read/Write Cycle (IOR/IOW):** The processor interacts with I/O devices.
- **Interrupt Acknowledge Cycle (INA):** The processor acknowledges an interrupt request.

Each of these cycles is characterized by a distinct combination of control and status signals that indicate the nature of the operation being performed.

The control unit in the 8085 is responsible for generating a set of control signals that regulate data transfer and communication with memory and I/O devices. These control signals include:

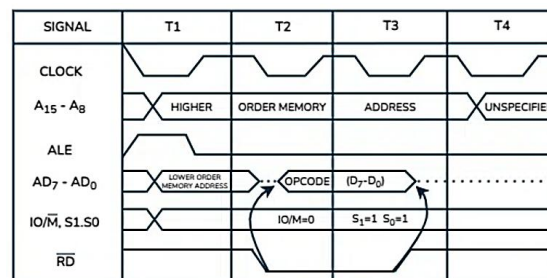
- **ALE (Address Latch Enable):** This signal is used to demultiplex the lower-order address and data bus (AD0–AD7). When ALE is high, the address is present on the multiplexed lines.
- **$\overline{RD}$ (Read):** A low signal indicates that the processor is reading data from memory or an I/O device.
- **$\overline{WR}$ (Write):** A low signal indicates that the processor is writing data to memory or an I/O device.
- **$IO/\overline{M}$:** This signal differentiates between memory and I/O operations. A low value indicates a memory operation, while a high value indicates an I/O operation.
- **S1 and S0:** These are status signals used in combination to indicate the current state or type of machine cycle in progress.

The correct interpretation and combination of these signals ensure that the memory and peripheral devices can identify the processor's intent and act accordingly.

The timing and control system in the 8085 is crucial for maintaining synchronization between the CPU and the external world. It ensures that data is read from or written to the right locations at the correct time. For example, if the $\overline{RD}$ signal is not activated properly during a memory read cycle, the microprocessor may receive incorrect data. Similarly, if the address and data are not latched correctly, it can lead to unpredictable behavior. Therefore, the timing diagrams, which graphically represent the relationships among various signals during different machine cycles, are used extensively for analyzing and designing systems based on the 8085 microprocessor.

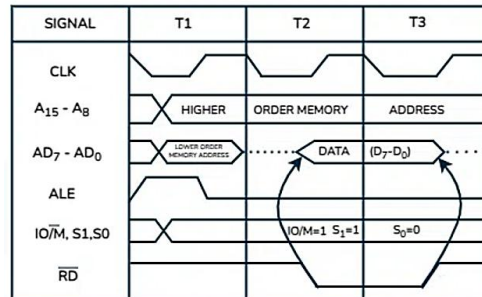
The Opcode Fetch cycle is the first step in the execution of any instruction. During this cycle, the 8085 microprocessor fetches the opcode (operation code) from memory to determine what operation to perform. This cycle takes 4 T-states (T1 to T4).

- T1: The microprocessor places the address of the instruction (stored in the Program Counter, PC) onto the address bus. It also sends the ALE (Address Latch Enable) signal to latch the lower byte of the address (A0–A7) using an external latch.
- T2: The $\overline{RD}$ (Read) signal is activated (low), enabling the memory to place the opcode on the data bus.
- T3: The microprocessor reads the opcode from the data bus and deactivates the $\overline{RD}$ signal.
- T4: Internal decoding of the opcode takes place. This step is critical to decide whether additional machine cycles are needed (e.g., to fetch operands or perform memory access).
- This cycle is essential for understanding what operation to perform next and always occurs at the beginning of instruction execution.



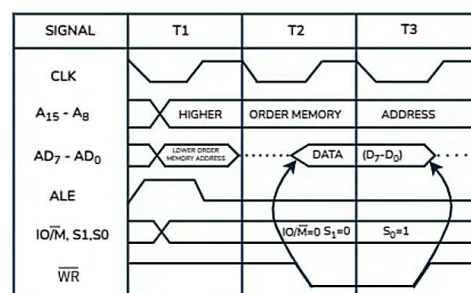
The Memory Read cycle is used when the 8085 needs to read data (or an operand) from memory, other than during the opcode fetch. This cycle takes 3 T-states (T1 to T3).

- T1: The address of the memory location to be read is placed on the address bus. The ALE signal goes high to latch the lower address.
- T2: The $\overline{RD}$ signal goes low, prompting the memory to place the data on the data bus.
- T3: The data is read by the microprocessor, and the $\overline{RD}$ signal goes high again, ending the read operation.
- This cycle is typically used for fetching operands or reading from stack or data memory during instruction execution.



The Memory Write cycle is used to write data from the microprocessor to a specific memory location. This cycle also takes 3 T-states.

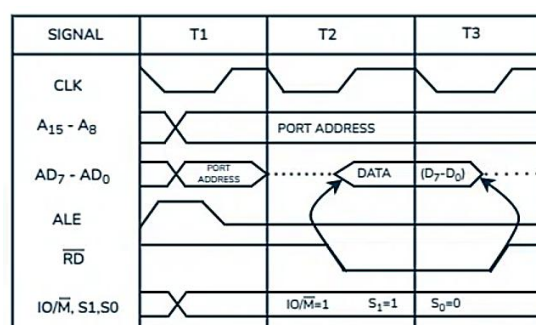
- T1: The address of the memory location to write to is placed on the address bus, and ALE is triggered to latch the lower address byte.
- T2: The data to be written is placed on the data bus, and the $\overline{WR}$ (Write) signal is made active (low).
- T3: The memory latches the data from the data bus, and the $\overline{WR}$ signal is deactivated, completing the write process.
- This cycle is used in operations like storing results in memory or pushing data onto the stack.



The I/O Read cycle is used to read data from a peripheral device (input device) using the I/O ports. It also consists of 3 T-states.

- T1: The 8-bit I/O port address is placed on both the lower and upper halves of the address bus (since only 8-bit addressing is used in I/O-mapped I/O). ALE is activated to latch the address.
- T2: The $\overline{RD}$ signal goes low, and the selected I/O device places data on the data bus.
- T3: The microprocessor reads the data and deactivates the $\overline{RD}$ signal.

This cycle is used in instructions like IN to receive data from peripherals like keyboards or sensors.



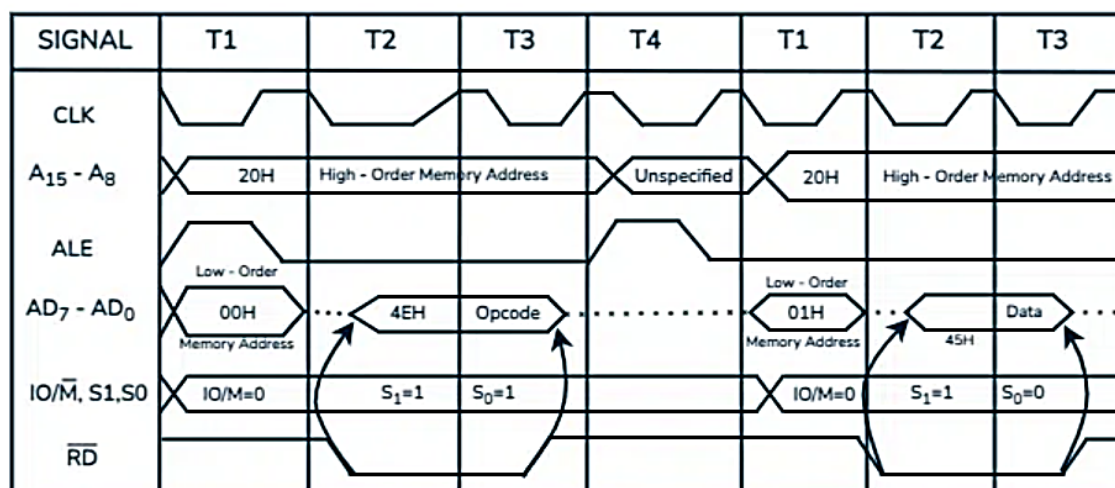
The I/O Write cycle is the counterpart to the I/O read and is used to send data from the 8085 to an output device. It also takes 3 T-states.

- T1: The I/O port address is placed on the address bus, and ALE is used to latch the address.
- T2: The data is placed on the data bus, and the $\overline{WR}$ signal is made active (low) to notify the I/O device that data is ready to be written.
- T3: The I/O device latches the data, and the $\overline{WR}$ signal goes high again, ending the cycle.
- This cycle is employed in instructions like OUT to send data to peripherals such as displays, printers, or actuators.

5.12 Instruction cycle

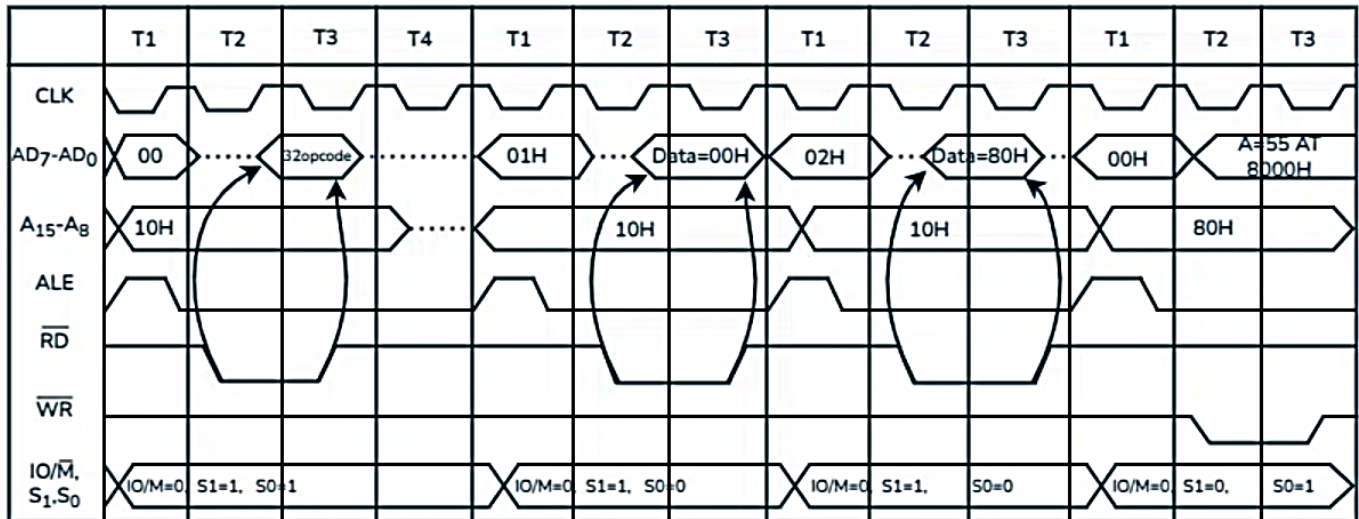
Timing Diagram of MVI A, 45

- Loading opcode 4EH from memory 2000H. (Operation Code Loading Machine Cycle)
- Read (move) data 45H from memory 2001H. (memory reading).



Timing Diagram of STA 8000H

- STA stands for Store Accumulator - the contents of the accumulator are stored at (1000H).
- The transaction code of STA indicates 32H. It is provided by 1000H memory (see picture). - In the machine cycle.
- Then read the sub address (00). - Memory reading machine cycle
- Read higher memory location (80) – Memory read machine cycle.
- Consider the combination of the two addresses and write the details of the accumulator for 8000. Assume the memory address for the instruction and let the content of accumulator is C7H. So, C7H from accumulator is now stored in 8000H.



5.13 Memory- Reference Instructions

Memory-reference instructions are machine instructions that involve accessing memory locations.

Instruction	Meaning
AND	Logical AND between AC and memory, result in AC
ADD	Add content of memory to AC
LDA	Load content of memory into AC
STA	Store content of AC into memory
BUN	Branch unconditionally
BSA	Branch and Save Return Address
ISZ	Increment and Skip if Zero

5.14 Register Reference Instructions

Symbol	Meaning	Operation Performed
CLA	Clear AC	$AC \leftarrow 0$
CLE	Clear E	$E \leftarrow 0$
CMA	Complement AC	$AC \leftarrow AC'$ (bitwise complement)
CME	Complement E	$E \leftarrow E'$

CIR	Circulate right (rotate AC + E right)	$(AC, E) \leftarrow (AC, E) \text{ shifted right}$
CIL	Circulate left (rotate AC + E left)	$(AC, E) \leftarrow (AC, E) \text{ shifted left}$
INC	Increment AC	$AC \leftarrow AC + 1$
SPA	Skip next instruction if AC is positive	If $AC > 0$, skip next instruction
SNA	Skip next instruction if AC is negative	If $AC < 0$, skip next instruction
SZA	Skip next instruction if AC is zero	If $AC = 0$, skip next instruction
SZE	Skip next instruction if E is zero	If $E = 0$, skip next instruction
HLT	Halt	Stop the computer

5.15 IO Reference Instructions

Symbol	Meaning	Operation Performed
INP	Input character to AC	$AC(0-7) \leftarrow INPR, AC(8-15) \leftarrow 0$
OUT	Output character from AC	$OUTR \leftarrow AC(0-7)$
SKI	Skip next instruction if input flag is set	If $IF = 1$, skip next instruction
SKO	Skip next instruction if output flag is set	If $OF = 1$, skip next instruction
ION	Turn interrupt system on	Enable interrupts ($IEN \leftarrow 1$)
IOF	Turn interrupt system off	Disable interrupts ($IEN \leftarrow 0$)

5.16 Interrupt

In the Intel 8085 microprocessor, interrupts are special signals that temporarily halt the normal execution of a program so the processor can respond to an urgent task or event. Once the task is addressed, the microprocessor resumes the original program. Interrupts are crucial in real-time systems where the processor needs to quickly respond to external or internal events. The 8085 microprocessor supports five hardware interrupts, and these can be broadly classified into two main categories: vectored and non-vectored, and maskable and non-maskable interrupts. Let's examine them in detail.

1. Vectored and Non-vectored Interrupts

Vectored Interrupts are those in which the address of the Interrupt Service Routine (ISR) is predetermined or hard-coded within the microprocessor. The 8085 has four vectored interrupts: TRAP, RST 7.5, RST 6.5, and RST 5.5. Each of these is automatically transferred to a fixed memory location when the interrupt occurs:

TRAP → 0024H

RST 7.5 → 003CH

RST 6.5 → 0034H

RST 5.5 → 002CH

Non-vectored Interrupts, in contrast, require the external device to supply the address of the ISR. The 8085 has one non-vectored interrupt: INTR (Interrupt Request). When the INTR signal is received and acknowledged, the microprocessor expects the external device to place the ISR address on the data bus.

2. Maskable and Non-maskable Interrupts

Maskable Interrupts are those that can be enabled or disabled (masked) through software instructions. These include RST 7.5, RST 6.5, RST 5.5, and INTR. The programmer can selectively enable or disable these interrupts using the EI (Enable Interrupt) and DI (Disable Interrupt) instructions, as well as through the Interrupt Enable (IE) and Interrupt Mask (SIM) instructions.

Non-maskable Interrupts cannot be disabled by the programmer. The only non-maskable interrupt in 8085 is TRAP. It is also edge and level sensitive, meaning it recognizes a rising edge and then stays active as long as the signal remains high. Due to its non-maskable nature and high priority, TRAP is typically used for critical events such as power failure or emergency shutdown.

3. Individual Interrupts Explained

TRAP: This is the highest priority interrupt. It is both edge- and level-triggered, making it very reliable and suitable for emergency operations. Being non-maskable, it always gets executed when triggered.

RST 7.5: This is a maskable, edge-triggered interrupt. It has the second-highest priority after TRAP. Since it responds to a rising edge, it only activates when the signal transitions from low to high.

RST 6.5 and RST 5.5: These are maskable and level-triggered interrupts. RST 6.5 has higher priority than RST 5.5. A high signal level must be maintained for the interrupt to be recognized.

INTR: This is the lowest priority interrupt. It is also maskable and requires external hardware to provide the ISR address. It is typically used for general-purpose interrupts in which precise ISR locations are managed by the programmer.

Summary of Priority

The interrupts in 8085 follow this priority order (from highest to lowest):

TRAP > RST 7.5 > RST 6.5 > RST 5.5 > INTR

5.17 Design of Accumulator Unit

The accumulator unit is a fundamental component in many processor architectures, especially in simple or early CPU designs, where it plays a critical role in arithmetic and logic operations. It is essentially a specialized register that temporarily stores intermediate results produced during the execution of instructions. A well-designed accumulator unit includes several key components: the accumulator register, an Arithmetic Logic Unit (ALU), an input multiplexer (MUX), control signals, and optionally a flags register.

Accumulator Register

At the core of the accumulator unit is the accumulator register, which stores the current value used or produced during computation. It serves as the central repository for input values, intermediate results, and final outputs of operations. For instance, when the CPU executes an instruction to add two numbers, one operand is often stored in the accumulator, while the other may be fetched from memory or another register. After the operation, the result replaces the previous value in the accumulator. This register is typically a high-speed storage location that interacts directly with the ALU.

Arithmetic Logic Unit (ALU)

The ALU is responsible for executing arithmetic (e.g., addition, subtraction) and logic operations (e.g., AND, OR, NOT) on the data contained in the accumulator and other sources. When a command is issued to perform an operation, the ALU takes input from the accumulator and a second source (another register or memory location), computes the result, and returns it—usually back to the accumulator. The design of the ALU ensures that it can perform a wide range of functions, and its tight integration with the accumulator allows for streamlined instruction execution.

Input Multiplexer (MUX)

The input multiplexer (MUX) plays a crucial role in determining the source of data that will be loaded into the accumulator. Since the accumulator can receive inputs from multiple sources—such as main memory, a data bus, or the ALU result—a MUX is used to select one of these sources at a time based on control signals. This flexibility allows the processor to handle various types of instructions and data routing scenarios efficiently.

Control Signals

Control signals orchestrate the behavior of the accumulator unit. These signals include commands such

as Load (to load new data into the accumulator), Clear (to reset its contents to zero), and Hold (to retain the current value without change). These actions are triggered by the CPU's control unit, which decodes instructions and sends appropriate signals to each component. Control logic ensures that the accumulator performs the correct operation at the correct time, maintaining synchronization with the instruction cycle.

Flags Register (optional)

An optional but valuable addition to the accumulator unit is the flags register, which stores status bits that reflect the outcome of operations performed by the ALU. Common flags include the Zero flag (Z), which is set when the result of an operation is zero; the Sign flag (S), indicating whether the result is positive or negative; and the Carry flag (C), which indicates an overflow in arithmetic operations. These flags provide critical feedback to the control logic, influencing conditional branches and other decisions in program flow.

In conclusion, the accumulator unit is a compact but powerful subsystem within the CPU, enabling efficient execution of arithmetic and logical operations. Through its core components—the accumulator register, ALU, MUX, control signals, and optional flags—it supports a streamlined and versatile computational model, especially suitable for simple instruction sets or embedded processors.

Component	Function
Register (Accumulator Register)	Stores the current value.
ALU (Arithmetic Logic Unit)	Performs operations like ADD, SUB, AND, etc.
Input Multiplexer (MUX)	Chooses input for the accumulator: memory data, bus data, or ALU result.
Control Signals	Manage actions: Load, Clear, Hold.
Flags Register (optional)	Updates status flags: Zero (Z), Sign (S), Carry (C), etc.

Parul[®] University | **NAAC** **A++**
Vadodara, Gujarat | **GRADE**



<https://paruluniversity.ac.in/>